



Mastère Data Engineer

Titre RNCP 39586

Ingénieur en Science des Données – Spécialité

Infrastructure Data



RENDU BLOC 1 – Collecter Transformer et Sécuriser les données

Août 2025

**Sujet : Conception d'une Architecture Data pour
Analyser les Consommations d'Energie
Réelles et la fiabilité des DPE des logements en France**

Réalisé par :

GBENOU Merveille Féréol Mahugnon

Sommaire

Figures	2
A - Collecte de données	3
1- Problématique et identification des sources de données	3
2- Définition de la stratégie de collecte de données	4
3- Exemple de collecte de données	8
4- Méthode d'automatisation de collecte	10
B - Stockage des données structurées et non structurées	12
1- Stratégie de stockage des données	12
2- Système de stockage des fichiers	12
3- Base de données et modèle de données	13
C - Structuration, transformation et enrichissement des données	13
1- Outils et des technologies de traitement des données	13
2- Étapes de transformations, méthodes et outils utilisés	14
3- Processus ETL - déploiement d'un processus ETL	14
4- Automatisation et orchestration du traitement des données	16
D - Sécurisation des données	16
1- Présentation des enjeux de sécurité des données	16
2- Politique de sécurité des données	17
3- Schéma d'architecture de sécurité	18
Sources	21
Annexes	21

Figures

FIGURE 1 : APERÇU DES DONNEES OBTENUES VIA L'API ENEDIS	6
FIGURE 2 : NOMBRE MOYEN DE PDL ENEDIS PAR CODE DEPARTEMENT	8
FIGURE 3 : URLS D'EXTRACTION DES DONNEES DEPUIS LES APIS	10
FIGURE 4 : APERÇU RESULTAT DE LA COLLECTE	10
FIGURE 5 : CODE SNIPPET - PIPELINE EXTRACTION, METHODE EXTRACT	11
FIGURE 6 : ORDONNANCEMENT DES TACHES DANS LA COLLECTE	12
FIGURE 7 : ÉTAPES DE TRANSFORMATION DES DONNEES	14
FIGURE 8 : ORCHESTRATION DU JOB ETL	16
FIGURE 9 : FLUX D'ECHANGE DE DONNEES ENTRE LES SERVICES	19
FIGURE 10 : SECURITE DE LA BASE DE DONNEES – AUTHENTIFICATION PAR TOKEN	20
FIGURE 11 : APERÇU RESULTAT DE LA COLLECTE EN MODE BATCH (+ DE 10_000 PDL ENEDIS)	22
FIGURE 12 : SCHEMA UML DU FONCTIONNEMENT DE L'ETL	22
FIGURE 13 : STRATEGIE DE MISE A JOUR, REPRISE SUR ERREUR ET IDEMPOTENCE	23
FIGURE 14 : SCHEMA DE LA BASE DE DONNEES - ZONE GOLD	23
FIGURE 15 : CODE SNIPPET - TACHE COLLECTE DE DONNEES ENEDIS	24
FIGURE 16 : CODE SNIPPET- TACHE REQUETE DE GEOCODING BAN	24
FIGURE 17 : CODE SNIPPET - TACHE REQUETE API ADEME	25
FIGURE 18 : CODE SNIPPET - TACHES DE JOINTURES, EXTRACTION	25
FIGURE 19 : DIAGRAMME DE CAS D'UTILISATION PACKAGE ETL	26
FIGURE 20 : SWAGGER UI DU SERVEUR API	26
FIGURE 21 : SWAGGER UI, ROUTES READ DATA	27
FIGURE 22 : ORCHESTRATION DU JOB ETL AVEC DEPLOIEMENT LOCAL	27

A - Collecte de données

1- Problématique et identification des sources de données

Rappel problématique et objectifs

Le Diagnostic de Performance Énergétique (DPE) est un document qui évalue la consommation énergétique d'un bâtiment et son impact sur l'environnement. Il attribue une étiquette allant de A à G pour chaque logement ou bâtiment, qui évalue sa consommation d'énergie d'une part (étiquette DPE) et son impact en termes d'émission de gaz à effet de serre (étiquette GES) d'autre part. Ceci permet d'identifier les logements passoires énergétiques c'est-à-dire ceux qui consomment le plus d'énergie (électricité, gaz, fioul, énergies renouvelables etc..) ou émettent le plus de gaz à effet de serre. Aujourd'hui, il y a une seule méthode de calcul est utilisée et reconnue. Il s'agit de la méthode 3CL qui analyse l'isolation, les systèmes de chauffage et d'autres caractéristiques du bâtiment notamment les matériaux de construction. Elle se base sur ces éléments pour estimer les consommations énergétiques du chauffage, eau chaude, climatisation, éclairage et usages auxiliaires. Cette méthode remplace la précédente (depuis juillet 2021) qui se basait uniquement sur les factures d'énergie c'est-à-dire les consommations réelles. Bien qu'étant la plus utilisée, la méthode 3CL repose sur des calculs standardisés qui ignorent les modes de vie et taux d'occupation réels, pouvant créer des écarts significatifs avec les consommations effectives. Ainsi, historiquement, les méthodes de calcul du DPE présentent des imprécisions importantes dans l'estimation des consommations énergétiques réelles. Avec l'accélération de la transition énergétique et le déploiement des dispositifs d'aide publique, il devient essentiel d'exploiter le plus de données disponibles pour développer des modèles prédictifs performants et optimiser l'accompagnement des ménages dans leurs projets de rénovation.

C'est dans ce contexte que Enedis, en partenariat avec l'Open Data University (ODU) ont ouvert un défi intitulé : « *Évaluer l'impact de la classe de DPE sur les consommations électriques de logements et confronter les estimations issues des DPE à des mesures réelles* ». Le but du défi, est de mettre en place une plateforme pour éclairer et motiver les acteurs de la transition écologique dans leur prise de décisions en mettant à leur disposition des analyses statistiques et des indicateurs clés. Concrètement, ce défi peut être décomposé en plusieurs sous questions business dont les deux suivantes.

- Combien gagne-t-on sur ses factures d'électricité en euros chaque année à améliorer sa classe de DPE ?
- Les estimations de consommation électriques conventionnelles fournies dans les DPE reflètent-elles bien la réalité mesurée des consommations électriques et quelle est la variabilité restante due à l'hétérogénéité des modes de vie et au taux d'occupation ?

L'objectif global de ce projet, sur la base de ces deux problématiques, est de concevoir un système d'information qui permettra :

- D'éclairer les décisions de rénovation en quantifiant les gains potentiels associés à une amélioration de la performance énergétique d'un logement (amélioration de l'étiquette DPE).
- De se prononcer sur la conformité au réel des estimations de consommation électrique présentées dans les DPE par des comparaisons aux données de consommations réelles et de quantifier la partie de la consommation électrique variable due aux habitudes des personnes.

Identification et justification des sources de données

Le système d'information sera organisé autour des données sur la consommation électrique réelles fournie par Enedis et complétée par la Base Nationale des Adresses (BAN) et les bases de l'Agence pour la Transition Écologique (ADEME). Le SI développé à cet effet a pour nom *ENEDIS_ADEME_VOLT-DPE* (Validation Of Linky versus Theorical DPE). En effet, l'analyse du besoin montre que pour répondre à cette problématique, il faudrait disposer de workflows data pour récupérer :

- Les consommations d'électricité réelles collectées par adresses fournies par Enedis
- Les adresses normalisées fournies par la Base Nationale des Adresses
- Les caractéristiques des logements (matériaux, types d'énergies utilisé, surface etc..) fournies par l'ADEME dont les DPE

La veille technologique et l'exploration de la documentation ont révélé que des API REST existent et exposent ces données. Ainsi les sources de données identifiées sont les suivantes :

- API Enedis
- API de la Base Nationale des Adresses
- API de l'ADEME

La stratégie de collecte portera donc sur la manière de requêter et industrialiser le requêtage de ces données avec des pipelines ETL.

2- Définition de la stratégie de collecte de données

Objectifs de la collecte

L'objectif de la collecte est de construire une base de données complète en regroupant les données de consommation réelles des logements français et leurs caractéristiques (matériaux de construction, diagnostic DPE etc..). En effet, la stratégie de veille a révélé qu'il n'existe actuellement aucune source de données unifiée disponible en libre accès nous permettre de réaliser cette étude. Les données nécessaires pour répondre à la problématique ont été énumérées ci-dessus. Elles sont collectées depuis 3 sources différentes pour construire le jeu de données. Il s'agit des APIs exposées par Enedis, la BAN et par l'ADEME. Cette stratégie a été adoptée car ces données sont disponibles en accès libre (open data), maintenus et mis à jour par des organismes publics.

Stratégie de la collecte

Il faut mettre en place un module pour extraire les données de ces 3 sources. Les données de consommations d'énergie réelles fournies par Enedis seront associées aux logements et leurs caractéristiques (collectées par l'ADEME). Les adresses seront utilisées comme clés de jointure mais puisque les adresses peuvent différer selon l'orthographe, elles seront d'abord normalisées via l'API de la BAN. Les identifiants obtenus serviront alors de clés de jointure.

Le module de collecte constitue la première partie du pipeline ETL qui est complété par le module de transformation (sur la base de l'exploration des données) et du chargement dans un système de stockage pour la persistance des données. Plus précisément, le module collecte permet à lui seul d'obtenir le jeu de données complet qui sert la partie analytique et exploratoire. C'est cette dernière qui permet de sortir les colonnes pertinentes à exposer sur notre serveur de données. Les étapes de d'exploration seront industrialisées pour être intégrées à l'ETL afin d'obtenir des données propres au bout du processus.

Enfin, le long du processus, les fichiers de données intermédiaires et les fichiers de logs sont stockés dans un S3 (serveur de stockage de fichiers bruts hébergé). Le processus d'ETL reprend l'architecture médaillon bronze, silver et gold. Pendant ce temps en arrière-plan un orchestrateur gère l'ordonnancement des tâches et des logs sont envoyés sur le serveur l'orchestrateur.

Le schéma de la Figure 12: Schéma UML du fonctionnement de l'ETL donne un aperçu complet de la stratégie de collecte et des deux autres étapes.

Contraintes

Techniquement, ce projet consiste à fusionner les DPE et caractéristiques des logements et les données de consommation électrique d'Enedis par adresse pour obtenir un jeu de données complet exploitable. Cette fusion repose sur les adresses comme clé d'appariement, et implique la normalisation des adresses via l'API de la BAN. En termes de volumétrie à l'échelle de la France, environs 6 millions de logements seront traités pour les DPE ADEME et 400 000 adresses (points de livraisons) par an pour les consommations électriques d'Enedis à chaque mise à jour de la base. Les formules utilisées pour estimer le volume de données et les détails des calculs figurent en Annexe. En somme, le SI aura à ingérer, traiter et exposer (à l'échelle de la France métropolitaine) entre 15 et 20 Gb de nouvelles données brutes par an ou à l'initialisation. Ensuite chaque semaine, il y a quelques méga octets à traiter car la base de l'ADEME est mise à jour chaque semaine. Pour cette raison, les contraintes techniques identifiées portent sur la scalabilité de la solution proposée (capacité à supporter la montée en charge et garder un temps de réponse correct). L'équipe fera donc particulièrement attention à optimiser la base de code et choisir des solutions pour chaque service en prenant en compte ces aspects.

Une autre contrainte de ce projet porte sur le budget. En effet, le projet est développé dans le cadre d'un challenge universitaire (Open Data University) sur l'année universitaire 2024-2025. Le coût de mise en place de la solution devra être quasi nul car le budget que l'équipe pourra consacrer à la réalisation de ce projet est fortement limité.

Enfin la dernière contrainte a été mentionnée dans le contexte du projet. Il n'existe pas à ce jour une source de données unifiée et complète pouvant servir de base à ce projet. Nous sommes dans l'obligation de développer notre propre solution de collecte de données à partir des sources open data en restant open source nous-mêmes (sur la partie de la collecte) pour permettre aux futurs chercheurs d'aborder le projet plus facilement, voire reproduire plus facilement les résultats des analyses.

Présentation des sources de données

- API Enedis

Plus précisément il s'agit du jeu de données *consommation-annuelles résidentielle par adresse* exposé par cette API. Ce jeu de données présente la consommation électrique annuelle des adresses par lots de 10 logements ou plus. En effet, un lot de logements correspond à un point de livraison agrégé (ou PDL) de type résidentiel. Enedis expose les données ainsi de manière agrégée en raison de la nature personnelle des données. Autrement, les adresses et les habitudes de consommation personnelles seraient divulguées ce qui entraînerait des infractions au RGPD. Malgré cette contrainte, nous pouvons mener les analyses avec ces données de consommations agrégées.

Pour le requêtage, la version 2.1 de l'API (la plus récente) est celle qui sera utilisée comme source. Nous disposons à cet effet des données de consommation par adresses agrégées par année, par code de département et par code commune. La base est mise à jour de manière annuelle. Avec l'API, les jeux de données sont accessibles en format json (limité à 10_000 observations) ou en fichier .csv téléchargeable pour une extraction en batch dans le cas où plus de 10_000 lignes sont nécessaires.

	0	1	2
annee	2022	2022	2022
code_iris	060290102	060290111	060290117
nom_iris	La Source	Croisette-Palm-Beach	Le Vieux Port-Les îles
numero_de_voie	13	20	23
indice_de_repetition	None	None	None
type_de_voie	RUE	RUE	AVENUE
libelle_de_voie	LACOUR	LATOUR MAUBOURG	LAUGIER
code_commune	06029	06029	06029
nom_commune	Cannes	Cannes	Cannes
segment_de_client	RESIDENTIEL	RESIDENTIEL	RESIDENTIEL
nombre_de_logements	60	110	57
consommation_annuelle_totale_de_l_adresse_mwh	70.96	251.986	94.023
consommation_annuelle_moyenne_par_site_de_l_adr...	1.183	2.291	1.65
consommation_annuelle_moyenne_de_la_commune_mwh	2.96	2.96	2.96
adresse	13 RUE LACOUR	20 RUE LATOUR MAUBOURG	23 AVENUE LAUGIER
code_epci	200039915	200039915	200039915
code_departement	06	06	06
code_region	93	93	93
tri_des_adresses	60015	60023	60028

Figure 1: Aperçu des données obtenues via l'API Enedis

- API de la Base Adresse Nationale

La BAN expose une API de géocodage qui permet de normaliser les adresses brutes en établissant des correspondances avec la base. Avec cette source de données, une orthographe normalisée des adresses brutes présentent dans les bases de Enedis et de l'ADEME peuvent être obtenues avec un identifiant et un score de confiance sur le géocodage. La construction de la BAN est assurée par les communes au niveau local accompagnées par l'Agence Nationale de la Cohésion des Territoires (ANCT). L'exploitation et la diffusion de la BAN sont pilotées par l'Institut National de l'Information Géographique et forestière (IGN). Tout comme l'API Enedis, la BAN est également accessible en open data dans le cadre de la licence ouverte Etalab (voir Annexe). Les données sont accessibles sous forme de flux (format json) et de fichiers (téléchargement en batch, plusieurs adresses à géocoder).

- API de l'ADEME

Il s'agit plus précisément du jeu de données *DPE-logements-depuis-2021*. En effet, l'ADEME est chargée de collecter les DPE réalisés par les diagnostiqueurs dans le cadre de la location ou de la vente d'un logement depuis 2013. A cet effet elle expose les DPE réalisés via une API REST. Le DPE contient des informations sur les caractéristiques du bâtiment ou du logement (surface, orientation, murs, fenêtres, matériaux, etc.) ainsi que sur ses équipements (de chauffage, de production d'eau chaude sanitaire, de ventilation, etc.). Il a fait l'objet d'une réforme entrée en vigueur le 1er juillet 2021 portant sur la mise à jour de la méthode de calcul vers la méthode 3CL (présentée dans le contexte du projet). Son objectif est de fiabiliser le DPE afin d'en faire le dispositif central dans la réglementation liée à la rénovation énergétique. C'est pour cette raison que nous utilisons ce jeu de données.

Il existe une autre source de données pour les logements avant 2021 mais ils contiennent les DPE avec l'ancienne méthode et ne prennent pas en compte la réforme de 2021. En outre, des métadonnées sont disponibles avec ce jeu de données à savoir : un dictionnaire de données, une note technique précisant les informations essentielles à connaître afin d'interpréter correctement les données fournies, une description des contrôles de cohérence effectués sur les données des DPE et un modèle de données.

Évolution jeu de données ADEME et autre jeu de données ADEME envisagé

En effet, le jeu de données *DPE-logements-depuis-2021* est remplacé depuis le 17 juillet 2025 par le jeu de données *DPE-logements-existants-depuis-2021*. L'ancien jeu de données de l'ADEME ayant servi de base pour les analyses jusqu'ici sera décommissionné à partir du 1^{er} septembre. La revue de la documentation montre que l'ancienne base sera décommissionnée pour laisser place à 3 jeux de données (DPE logements neufs – DPE logements existants – DPE tertiaires) plus homogènes puisque toutes les catégories de logements étaient mixées. Les nouvelles sources de données incluront la date de dernière modification d'un DPE et seront mises à jour chaque semaine contre chaque mois actuellement. ***Ceci était un imprévu qui nous a obligé à implémenter une stratégie de migration avec une mise à jour du planning du projet.***

Autres pistes explorées et perspectives

Une veille technologique a permis d'identifier un projet open source sur GitHub, également disponible sous forme de package Python, qui permet d'estimer la consommation d'énergie d'un bâtiment en fonction de données météorologiques et des tarifs EDF. L'algorithme utilise les types de bâtiments, leur surface, leur étiquette DPE, ainsi que la température horaire et le prix de l'énergie correspondants sur une journée complète (par pas de temps d'une heure). En fournissant ces paramètres, l'utilisateur peut obtenir une estimation approximative du coût énergétique en euros par mètre carré pour des bâtiments situés dans la même ville. La différence principale avec ce que nous réalisons ici est que nous mettons à disposition une manière open source d'obtenir un jeu de données complet avant de procéder à l'analyse exploratoire et la modélisation ce qui n'est pas le cas pour la plupart des projets explorés. Nous nous concentrons sur la partie en amont (backend) du travail que cette personne a réalisé afin de rendre reproductible aisément les analyses futures.

3- Exemple de collecte de données

Interrogation des sources de données

Comme vu précédemment, les données sont collectées depuis les endpoints de 3 API REST différentes. Dans la suite nous procédons à une collecte de données sur ces API en détaillant leur fonctionnement en backend (exhaustivité, exactitude et cadre réglementaire) par la même occasion.

- API Enedis et API ADEME : requêtes ODSQL

Les endpoints des APIs de Enedis et de l'ADEME sont des API REST (protocole http). Elles utilisent en backend la syntaxe du langage *Open Data Soft Query Language* (ODSQL). En effet, ODSQL est un langage de requêtage courant utilisé par les API REST des services publics en France qui fournit une manière uniformisée de réaliser les opérations de projection avec des prédicats basés sur les mêmes mots clés. De plus les APIs utilisant ODSQL ne requièrent pas d'API key (la plupart du temps) car les données sont en open source.

Par ailleurs, pour garantir la sécurité de leurs infrastructures, les appels APIs vers ces deux sources sont limités. En effet, pour Enedis, les requêtes sont limitées à 100 documents en une observation avec un système de pagination permettant d'obtenir jusqu'à 10 000 pour une même combinaison de filtres. Pour l'ADEME un utilisateur ne peut pas effectuer plus de 600 requêtes par intervalle de 60 secondes et la vitesse de téléchargement totale est limitée à 2 Mb/s.

- Exhaustivité données Enedis collectées

Les données Enedis sont requêtées par département et par année dans l'ETL. Le graphique suivant montre qu'en combinant ces filtres, nous obtenons moins de 10 000 PDL Enedis pour la plupart des départements. Les données sont découpées de cette manière car l'API limite le nombre de documents qu'on peut extraire avec une seule requête. Avec ce découpage, nous pouvons obtenir les données d'un département complet en une ou deux requêtes. La figure 2 ci-dessous montre le nombre de PDL Enedis par département en moyenne par année entre 2018 et 2023. Ce graphique prouve que ce découpage est efficace mais sera limité seulement pour les départements 75, 92, 69, 93, 94, 06 et 78 où il y a entre 10 000 et 50 000 PDL Enedis.

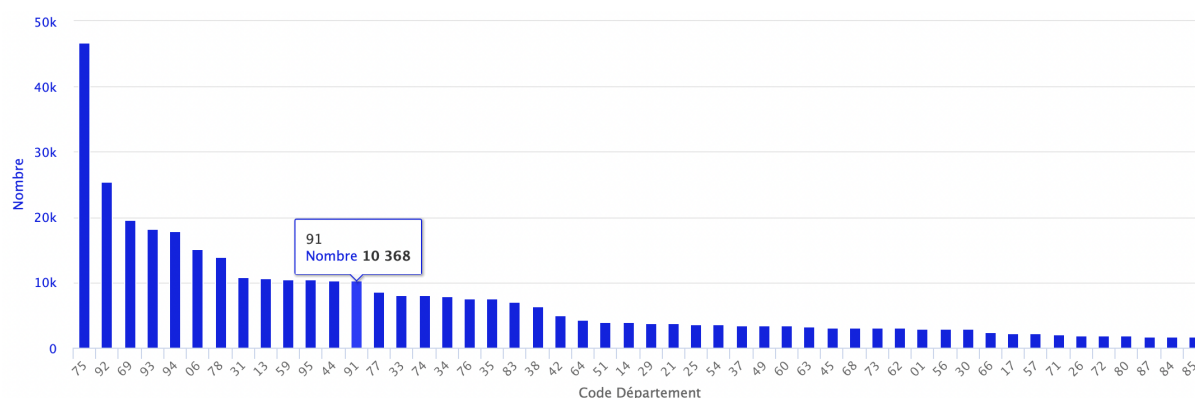


Figure 2 : Nombre moyen de PDL Enedis par code département

- Exactitude des données Enedis

Les données collectées sont agrégées par années et correspondent aux consommations d'électricité réelles des ménages sur ces périodes. Mais elles sont agrégées par une moyenne sur l'adresse et que pour une adresse donnée il y a entre 10 et 1 000 logements (donnée par la base de l'ADEME). Donc nous ne disposons pas de la consommation au niveau du logement (appartement ou maison) mais seulement de la moyenne sur l'adresse du PDL à laquelle appartient le logement. L'hypothèse est que des logements qui sont à la même adresse ont en général des besoins de consommation similaires. Cette donnée peut être biaisée dans la mesure où elle ne prend pas en compte le taux d'occupation c'est pour cette raison que d'autres indicateurs seront calculés sur la base de cette mesure.

- Exhaustivité des données ADEME

Le DPE est obligatoire seulement pour une vente, une location ou à l'achèvement de toute nouvelle construction. Ainsi, tous les biens immobiliers ne sont pas dotés d'un DPE. A ce titre, la base de données DPE ne couvre pas l'ensemble des logements et elle n'en est pas totalement représentative.

- Exactitude données ADEME

Pour l'exactitude, les données mises à disposition sont les données brutes saisies par les diagnostiqueurs dans le cadre de l'article L134-4-2 du code de la construction et de l'habitation. L'ADEME ne les retraite pas, donc elle n'est pas responsable de la qualité des données qui lui sont transmises. C'est plutôt le diagnostiqueur immobilier qui en est responsable. Toutefois, les données font l'objet de contrôles de cohérence. En cas de non-conformité, le DPE est rejeté si le contrôle est bloquant.

- API de la Base Adresse Nationale

La BAN expose une API de géocodage qui permet de normaliser les adresses brutes en établissant des correspondances avec la base. Avec cette source de données, une orthographe normalisée des adresses brutes présentent dans les bases de Enedis et de l'ADEME peuvent être obtenues avec un identifiant et un score de confiance sur le géocodage. Cette API présente certaines contraintes malgré le taux de disponibilité à 100%. En effet, un client (représenté par son adresse IP) est limité à 50 appels API par seconde en mode unitaire. Le géocodage en batch est disponible par extraction en fichier CSV à hauteur d'un appel simultané par adresse IP. Nous verrons dans la suite du document l'impact de ces limitations sur la capacité de l'ETL à collecter les données en masse. En ce qui concerne l'exhaustivité des données nous avons vu que la BAN est alimentée au niveau local par les communes et maintenue au niveau global par l'IGN et l'ANCT. Elle est réutilisée par de nombreux projets notamment pour la complétion automatique des adresses. En ce sens, elle est plutôt exhaustive comme source. Enfin pour l'exactitude, nous disposons d'un score de confiance associé au service de géocodage.

Mise en place de la technique de collecte : requêtes ODSQL

Les données sont collectées depuis ces 3 sources au moyen de requêtes utilisant le protocole http. Les url ODSQL sont préparées via des fonctions lambda prenant des

paramètres comme l'année, le code département ou l'identifiant BAN normalisé. La capture ci-dessous donne un aperçu du formatage des urls.

```

1 # générer une url ODSQL pour requêter l'api enedis
2 self.get_url_enedis=lambda annee, code_departement, limit, offset: f"https://data.enedis.fr/api/explore/v2.1/catalog/datasets/consommation-annuelle-residentielle-par-adresse/records?where\
3     =annee=date['annee'] and code_departement='{code_departement}'&order_by=tri_des_adresses&limit={limit}&offset={offset}"
4
5 # générer une url pour requêter l'api de la ban à partir d'une adresse
6 self.get_url_ademe_filter_on_ban = lambda key: f"https://data.ademe.fr/data-fair/api/v1/datasets/dpe03existant/lines?q_fields=identifiant_ban&q={key}" # update 19 juillet 2025
7 # générer une url pour requêter l'api de la ban à partir d'une adresse
8 self.get_url_ban_filter_on_adresse = lambda addr: f"https://data.geopf.fr/geocodage/search?q={addr}&limit=1" # adresse est complete car obtenue par concat dans enedis (update 23 juillet 2025)

```

Figure 3 : URLs ODSQL d'extraction des données depuis les APIs

Pour industrialiser la collecte des données, la logique de toute l'opération est encapsulée dans la classe *DataEnedisAdemeExtractor* que nous avons codé à cet effet dont le fonctionnement est repris par le graphique de la Figure 12 et présenté dans la rubrique Méthode d'automatisation de collecte. La fonction d'extraction principale est quant à elle présentée par la Figure 5 : Code snippet - Pipeline extraction, méthode extract.

Exemple de résultat de la collecte

Il faut importer le module *DataEnedisAdemeExtractor* qui se charge de la collecte depuis les APIs et fait les jointures entre les tables. Dans l'exemple ci-dessous, nous appelons avec les paramètres *code_departement*, *annee* et *rows* qui permettent de limiter les données aux logements de Paris en 2023. Le paramètre *rows* spécifie le nombre d'adresses PDL Enedis à extraire.

```

import time
from dpe_enedis_ademe_etl_engine.pipelines import DataEnedisAdemeExtractor

pipeline = DataEnedisAdemeExtractor()
pipeline.extract(code_departement=75, annee=2023, rows=3)

```

[16] ✓ 0.8s

... Environment: NOLOCAL

Le résultat de la collecte (221 colonnes et 20 logements après jointure avec la source ADEME) se présente comme ci-dessous pour les colonnes sélectionnées.

```

cols = [
    'identifiant_ban_ademe',
    'adresse_ban_ademe',
    '_geopoint_ademe',
    'code_commune_enedis',
    'consommation_annuelle_moyenne_par_site_de_l_adresse_mwh_enedis',
    'type_batiment_ademe'
]
pipeline.output(cols).drop_duplicates()

```

✓ 0.0s

	identifiant_ban_ademe	adresse_ban_ademe	_geopoint_ademe	code_commune_enedis	consommation_annuelle_moyenne_par_site_de_l_adresse_mwh_enedis	type_batiment_ademe
0	75118_0086_00021	21 Rue Affre 75018 Paris	48.88717103400885,2.355396985760465	75056	2.651	appartement
4	75118_0086_00021	21 Rue Affre 75018 Paris	48.88716995287075,2.355396624118101	75056	2.651	appartement
8	75113_0115_00004	4 Place Albert Londres 75013 Paris	48.82424302176755,2.363046056793319	75056	1.777	appartement
20	75117_0297_00011_b	11b Rue Ampère 75017 Paris	48.8851119710065,2.305407987721733	75056	4.187	appartement

Figure 4 : Aperçu résultat de la collecte (exemple)

4- Méthode d'automatisation de collecte

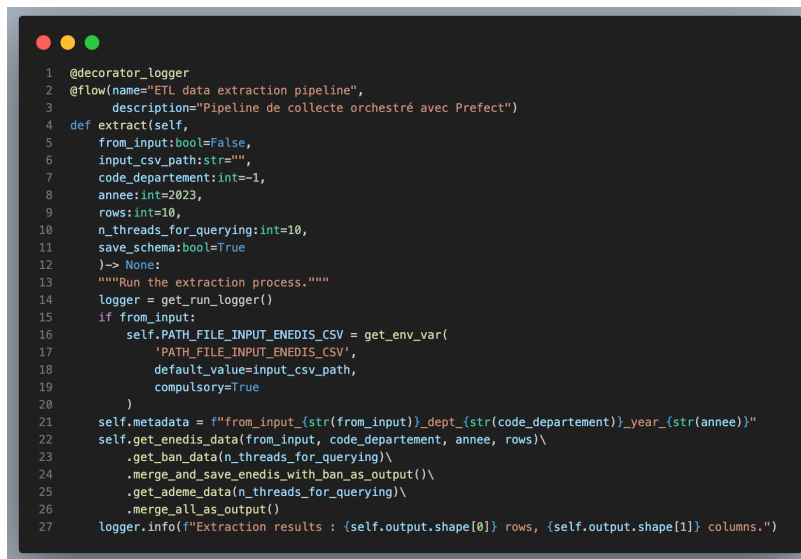
Les sources de données sont actualisées aux fréquences suivantes :

- Une fois par an pour Enedis : à l'année N+1 au mois de Juin (exemple en Juin 2025, nous disposons du jeu de données contenant les consommations d'électricité agrégées de 2024)

- En continu pour la BAN
- Une fois par semaine pour l'ADEME avec le nouveau jeu de données

Ces fréquences d'actualisation contraignent l'automatisation de la collecte. Pour cette raison, le pipeline d'extraction est planifié pour s'exécuter une fois par semaine afin d'ingérer les nouvelles données ADEME.

Le framework Prefect-flow est utilisé pour gérer l'orchestration et l'automatisation de la collecte des données. Ce choix a été fait car il possède plusieurs avantages dont l'intégration d'une interface de monitoring des pipelines, un système de logging intégré, un serveur pour gérer l'orchestration et surtout la mise en place est simple avec les décorateurs `@flow()` et `@task()`.



```

1  @decorator_logger
2  @flow(name="ETL data extraction pipeline",
3        description="Pipeline de collecte orchestré avec Prefect")
4  def extract(self,
5             from_input:bool=False,
6             input_csv_path:str="",
7             code_departement:int=-1,
8             annee:int=2023,
9             rows:int=10,
10            n_threads_for_querying:int=10,
11            save_schema:bool=True
12            )-> None:
13    """Run the extraction process."""
14    logger = get_run_logger()
15    if from_input:
16        self.PATH_FILE_INPUT_ENEDIS_CSV = get_env_var(
17            'PATH_FILE_INPUT_ENEDIS_CSV',
18            default_value=input_csv_path,
19            compulsory=True
20        )
21    self.metadata = f"from_input_{str(from_input)}_dept_{str(code_departement)}_year_{str(annee)}"
22    self.get_enedis_data(from_input, code_departement, annee, rows)\
23        .get_ban_data(n_threads_for_querying)\
24        .merge_and_save_enedis_with_ban_as_output()\
25        .get_ademe_data(n_threads_for_querying)\
26        .merge_all_as_output()
27    logger.info(f"Extraction results : {self.output.shape[0]} rows, {self.output.shape[1]} columns.")

```

Figure 5 : Code snippet - Pipeline extraction, méthode extract

Lorsque le `code_departement` vaut -1 on extrait tout, tout périmètre géographique confondu. Si le *nombre de lignes* vaut -1, on extrait le maximum de lignes possibles via l'API (10_000). Pour en extraire plus, il faut télécharger un fichier de données, le déposer dans un dossier et mettre le paramètre `from_input` à True du site ce qui n'est pas automatisable. En ce qui concerne l'optimisation, nous implémentons du multi threading pour appeler les APIs sources en prenant en compte leurs rate-limiteurs (Figure 15 à 17).

- API Enedis : 10 threads au max en concurrence
- API BAN : jusqu'à 10 threads en concurrence dans la limite de 30 requêtes/secondes car l'API a un rate limite de 50 requêtes/secondes
- API ADEME : 1 seul thread car le multi-threading ne fonctionne pas avec cette API

L'image ci-dessus montre les différentes étapes de la collecte dont la mécanique est décomposée en 7 tâches (3 tâches d'extraction, 2 tâches de validation de schéma des données et 2 tâches de jointure). L'ordonnancement se présente comme suit :

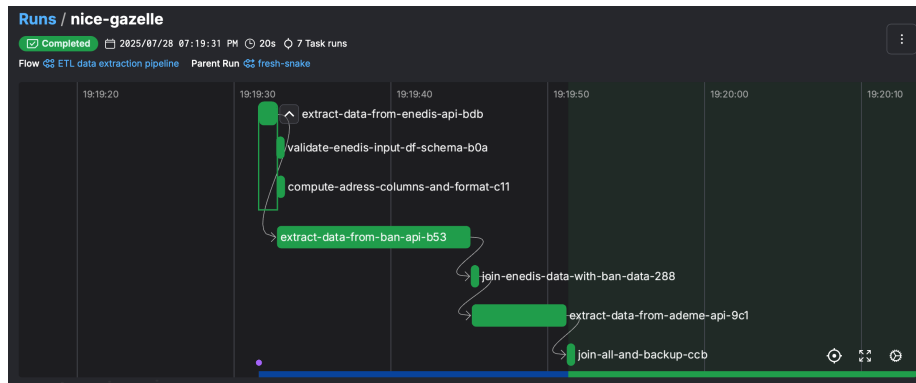


Figure 6 : ordonnancement des tâches dans la collecte

Ce pipeline de collecte fait partie du job d'ETL et il s'exécute chaque semaine pour intégrer les nouvelles données hebdomadaires exposées par l'ADEME. Pour un même ensemble d'inputs pour la fonction de collecte de données, le schéma de la Figure 13 présente la manière dont le pipeline de collecte gère les mises à jour, la reprise sur erreur et l'idempotence (obtenir les mêmes résultats pour un même ensemble d'inputs).

Enfin, le job de collecte est déployé sur un serveur prefect (local) grâce à la fonctionnalité de scheduling de prefect elle-même basé sur le framework croniter. Le cronjob s'exécute donc une fois par semaine à 9h les mercredis (Figure 8). Le déploiement est également réalisable sur un serveur distant hébergé par Prefect Cloud (freemium avec 1 instance de compte, 500H de compute serverless par mois et monitoring inclus).

B - Stockage des données structurées et non structurées

1- Stratégie de stockage des données

Les données traitées sont stockées sous forme de fichiers json et parquet pour garder un historique et sont également stockés sur le serveur de la base de données. Les fichiers intermédiaires de collecte sont stockés dans un stockage de type S3 filestorage et supprimés au bout de 3 mois au maximum. Les données finales sont stockées dans une base de données avec un schéma défini après transformations pour la persistance.

2- Système de stockage des fichiers

Les pipelines *DataEnedisAdemeExtractor* et *DataEnedisAdemeTransformator* que nous avons codés sont utilisés pour collecter et traiter la donnée héritent de la classe mère *FileStorageConnexion* (que nous avons codé également) qui gère la connexion au système de stockage de fichiers. Ce faisant, les pipelines de collecte et de transformation peuvent écrire et lire les fichiers depuis le filestorage. Le système de stockage de fichier utilisé ici est le S3 de AWS et le format des fichiers écrits et lus sont en parquet. Le filestorage choisi est S3 en raison du coût de mise en place nul avec la solution de développement MinIO (en conteneur docker) et de la simplicité de migration vers un stockage cloud AWS plus tard. En effet le client utilisé est boto3 qui permet une interopérabilité entre la solution de POC et le déploiement final. En ce qui concerne l'architecture, c'est le pattern médaillon qui est implémenté avec 3 zones de stockage : la zone bronze, la zone silver et la zone gold. L'utilité de chacune de ces zones est présentée par le schéma UML Figure 12: Schéma UML du

fonctionnement de l'ETL. Un aperçu de l'UI du filestorage utilisé pour le développement (MinIO) est présent sur les Figure 24 et 25.

3- Base de données et modèle de données

Pour la persistance, un serveur de données relationnel est ajouté au backend pour stocker les données propres. Cette solution de stockage a été choisie car d'une part le schéma des données à ce niveau est fixé et défini par le travail d'analyse et d'exploration qui a été réalisé sur les données silver. En effet, à l'étape des données silver, la table contient plus de 220 variables pas toutes pertinentes pour les utilisateurs finaux. D'autre part, en choisissant un schéma relationnel, les tables obtenues peuvent être requêtées avec le langage SQL. Les entités identifiées dans la modélisation sont : logements, adresses, villes, zone climatique et géocodage. La Figure 14 : Schéma de la base de données - zone gold donne un aperçu de la base qui est modélisée par un schéma relationnel en étoile.

Le schéma des données est composé de 5 entités principales connectées par des relations dont les cardinalités sont (1 à n). D'une part on distingue la table centrale "logements" constitue le cœur du modèle avec des attributs techniques incluant les identifiants de référence (id_ban, id_enedis, id_ademe), les caractéristiques de construction des logements et leurs données énergétiques. Cette entité agrège l'ensemble des informations nécessaires à l'évaluation de la performance énergétique des bâtiments. D'autre part, le modèle s'appuie sur des entités qui permettent de contextualiser les données logements et qui font office de dimension (axes de recherches). Il s'agit notamment de l'entité "adresses" qui fournit la localisation précise avec les compléments d'adresse, l'entité "ville" qui standardise les informations municipales (code postal, nom de ville). Les dimensions climatiques et géospatiales sont représentées par les entités "climat" (zone climatique) et "géocodage" (latitude, longitude, score de confiance du géocodage), toutes reliées par l'identifiant commun id_ban.

Enfin, en ce qui concerne la sécurité de la base, en plus du rôle administrateur, deux rôles sont créés au niveau de la base de données pour gérer les accès en écriture (pour l'application ETL) et en lecture (pour l'application de dashboarding).

C - Structuration, transformation et enrichissement des données

1- Outils et des technologies de traitement des données

La transformation des données se fait avec des scripts en langage python ce qui permet d'industrialiser le nettoyage à partir des étapes de l'analyse réalisée dans un notebook. En effet l'analyse se déroule dans la zone silver et les scripts de transformations automatisés permettent d'automatiser ces étapes pour passer à la zone gold.

Les données de la zone silver sont complètes mais pas propres. Nous avons une seule table avec plus de 220 variables et plus de 2 millions de lignes pour l'extraction de tous les départements sur l'année 2023. Le jeu de données brut est traité avec le langage Python au moyen d'un pipeline écrit par le Data Scientist de l'équipe (voir rubrique suivante pour les étapes). Ce choix a été fait car l'avantage de Python est qu'il met à disposition des frameworks spécialisées en traitement de la donnée comme Dask, Pandas, NumPy et Scipy.

Puisque les données sont inconnues au départ, il a fallu faire de l'exploration et Python offre une flexibilité pour les transformations non-standard (analyses statistiques avancées, imputation, ajout de métriques avec des cas particuliers et des visualisations pas forcément possibles avec des outils low-code). Enfin, l'écriture de scripts permet l'automatisation, le versionning et la traçabilité des transformations appliquées aux données.

Toutefois, Python a des limitations de mémoire et de performance sur de très gros volumes (notre cas ici puisque nous traitons jusqu'à 15Gb de données) au niveau de la zone silver. Pour faire face à cette limitation, nous utilisons le framework de traitement Dask pour le traitement distribué.

2- Étapes de transformations, méthodes et outils utilisés

Une partie de la transformation commence dans le pipeline d'extraction. Il s'agit des jointures entre les tables obtenues par collecte auprès des API sources. Les clés primaires et étrangères utilisées sont les *id_ban*. Une fois le dataset complet obtenu, pour passer de la zone silver à la zone gold (Figure 14 : Schéma de la base de données - zone gold), il y a un autre pipeline de 4 étapes qui déroule et qui correspond à la version automatisée des transformations effectuées à l'étape d'analyse. Il comprend les étapes de typage correct, imputation, calculs d'indicateurs et sélection de variables. Les variables finales à afficher sont sélectionnées par l'étape analyse/modélisation (les détails de l'analyse sont présentés dans le rendu du bloc 2).

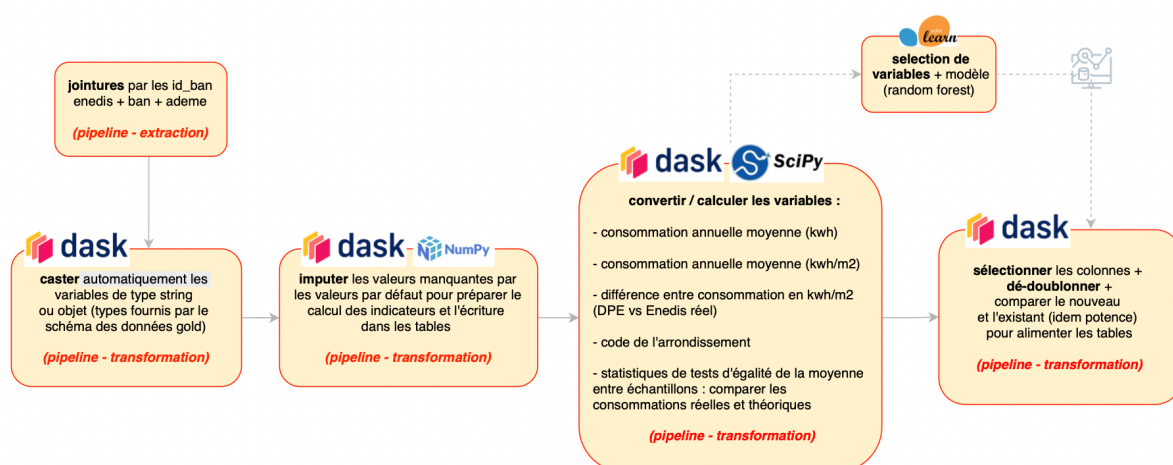
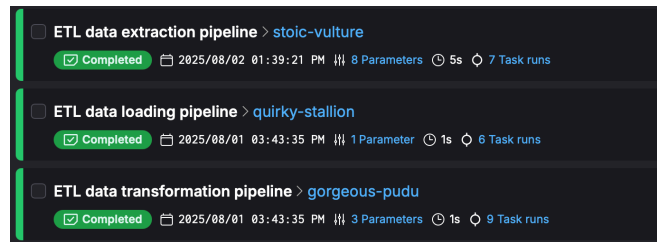


Figure 7 : Étapes de transformation des données

3- Processus ETL - déploiement d'un processus ETL

Les étapes de collecte, de traitement et de stockage de la donnée présentée jusqu'ici ont permis de construire le pipeline **dpe_enedis_ademe_etl_engine** qui est exposé sous forme de module/package dans notre SI et importé par le module de notre serveur API. Mais nous l'avons également déployé grâce à l'orchestrateur de Prefect qui est utilisé sur chacune des étapes.



Orchestration et déploiement : exécution 1 fois par semaine sur le serveur localhost (cf capture de l'UI - Figure 22)

Par ailleurs, le package **dpe_enedis_ademe_etl_engine** est également publié sur [l'index Pypi](#) et est spécialement conçu pour l'obtention d'un jeu de données pour l'analyse des performances énergétiques des bâtiments français. Il propose une architecture supportant à la fois l'exécution locale et l'exécution conteneurisée (intégrée à un ensemble de micro services, voir schéma de sécurité Figure 9), permettant des workflows de traitement de données adaptés à différents environnements d'exploitation. Le schéma (Figure 19) présente le diagramme des cas d'utilisation du module. Un utilisateur peut interagir avec différentes fonctionnalités du package ETL selon ses besoins :

- extraire et merger juste des données depuis les 3 sources de données présentées pour obtenir un dataset brut à analyser
- orchestrer et exécuter le pipeline complet avec les 3 étapes depuis l'extraction jusqu'au chargement dans le SGBDR (cf schéma UML)
- visualiser les logs de l'exécution de l'une ou l'autre activité dans les fichiers de trace ou le système de monitoring.

Le système de monitoring

Cette capacité de surveillance est importante car elle permet de suivre les performances et de résoudre les bugs. Le package intègre des fonctionnalités avancées de logging et de profilage (grâce au framework prefect utilisé pour l'orchestration) pour assurer une observabilité complète du pipeline de données. Les logs et les métadonnées des exécutions de l'ETL sont centralisés sur le serveur prefect, facilitant le monitoring en temps réel et le débogage. Le système génère automatiquement des métriques de performance et des statistiques d'exécution (durée d'exécution par fonction, tâche et ordonnancement), permettant l'optimisation des workflows et la détection des goulots.

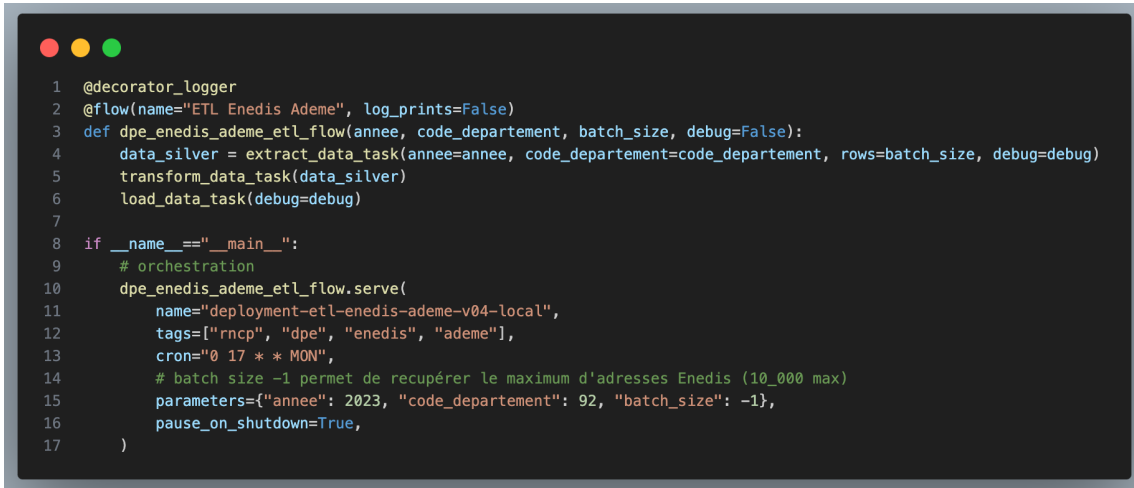
Configuration

La liste des variables d'environnement nécessaires sont présentes en Annexes et dans la documentation. Pour une utilisation locale, le package peut être installé directement depuis le repos avec la commande **pip install dpe_enedis_ademe_etl_engine**. Cette approche est idéale pour le requêtage de données à partir d'un fichier input (quand on a besoin de plus de 10_000 adresses Enedis) pour contourner la limite de leur API. L'utilisateur doit simplement configurer les variables d'environnement appropriées et définir les chemins locaux pour les différentes zones de données. En exécution locale, les fichiers intermédiaires sont stockés sur la machine hôte. En exécution distance, ils sont stockés sur le filestorage S3. Dans notre

SI, nous utilisons l'ETL comme module. Ceci permet de passer par l'API pour déclencher l'orchestration quand tous les autres micro-services sont démarrés.

4- Automatisation et orchestration du traitement des données

Les données ADEME sont actualisées chaque semaine donc le flow s'exécute chaque semaine grâce à l'orchestrateur prefect.



```
1 @decorator_logger
2 @flow(name="ETL Enedis Ademe", log_prints=False)
3 def dpe_enedis_ademe_etl_flow(annee, code_departement, batch_size, debug=False):
4     data_silver = extract_data_task(annee=annee, code_departement=code_departement, rows=batch_size, debug=debug)
5     transform_data_task(data_silver)
6     load_data_task(debug=debug)
7
8 if __name__=="__main__":
9     # orchestration
10    dpe_enedis_ademe_etl_flow.serve(
11        name="deployment-etl-enedis-ademe-v04-local",
12        tags=["rncp", "dpe", "enedis", "ademe"],
13        cron="0 17 * * MON",
14        # batch size -1 permet de récupérer le maximum d'adresses Enedis (10_000 max)
15        parameters={"annee": 2023, "code_departement": 92, "batch_size": -1},
16        pause_on_shutdown=True,
17    )
```

Figure 8 : Orchestration du job ETL

D - Sécurisation des données

1- Présentation des enjeux de sécurité des données

Les principales sources de données utilisées dans le cadre de ce projet proviennent de l'open data. Il s'agit de données accessibles à tous sans restriction technique, juridique ou financière. Par définition, ces données sont produites et exposées par des organismes publics et privés en l'occurrence ici Enedis, l'ANCT et l'ADEME. Les principes de l'open data sont le libre accès, le droit à la réutilisation et à la complétion pour en tirer de la valeur (à but lucratif ou non).

En ce qui concerne la responsabilité sur les données personnelles, les organismes qui exposent les sources d'open data procèdent au retraitement pour anonymiser à leur niveau les données pour ne pas se retrouver exposées aux sanctions de la CNIL. Puisque dans ce projet, les données sont fusionnées et transformées pour obtenir de nouvelles informations, nous faisons attention à ne pas désanonymiser ces données. Pour cette raison, les transformations effectuées sont uniquement des calculs arithmétiques.

Par ailleurs, dans ce projet, les données gold dans la base de données sont exposées au niveau du serveur (dont l'architecture est présentée à la Figure 9). Le point d'entrée du serveur est une API REST qui interface avec la base de données. Les failles de sécurité techniques potentielles auxquelles le serveur pourrait être exposé sont les suivantes :

- Les injections SQL : il s'agit de failles dans les routes de l'API permettant l'exécution de requêtes SQL en backend malveillantes via des inputs utilisateurs non validés. Ces

requêtes peuvent compromettre l'intégrité, la confidentialité et la disponibilité des données.

- Risque de fuite de données ou de réutilisation à des fins malveillantes
- Le risques de DDOS : requêtes multiples adressées à une même route

2- Politique de sécurité des données

Les lignes suivantes abordent le plan d'action relatif à la sécurité des données et décrivent la politique implémentée.

Comme présenté dans la rubrique de la collecte, les données de consommations Enedis sont agrégées par point de livraison (quartier, rue, immeuble) et donc à aucun moment des consommations individuelles des logements des Français ne seront exploitées. Ceci limite le risque d'identification des habitudes de consommation des personnes. Ensuite, la Base Nationale des adresses (BAN), dans sa forme actuelle ne communique pas de nom ni d'informations personnelles. Elle ne tombe pas sous le coup du RGPD car les adresses postales sont considérées comme des données publiques par nature car elles ne permettent pas, à elles seules, d'identifier directement une personne physique. Enfin, d'après la documentatin, l'ADEME est soumise au RGPD en tant qu'établissement public à caractère industriel et commercial qui collecte et traite des données à caractère personnel. De ce fait les données qu'elle expose sont limitées et anonymisées (aucune donnée d'identification ni de contact) après traitement conformément au RGPD.

En ce qui concerne notre SI, puisque nous collectons et nous traitons la donnée, nous veillons à ne jamais exposer la base de données complète au public ni le résultat complet du traitement. Les seules données de consommation exposées sont les consommation moyennes en kwh/m2 pour les logements. En outre, les étapes de transformation sont composées uniquement de calculs arithmétiques et dans le résultat final exposé au client du serveur, nous veillons à ne pas diffuser la liste des adresses ni des identifiants des logements (voir Figure 14 : Schéma de la base de données - zone gold). La table des adresses est accessible aux utilisateurs de niveau administrateur (équipe de développement). Les lecteurs voient seulement les données niveau villes, et les caractéristiques des logements.

Ensuite, pour prévenir les risques liés à l'intégrité de l'infrastructure et aux fuites des données identifiés dans la section 1 de cette partie, le plan d'action dans la conception de l'API consiste à :

- Implémenter un design pattern Data Access Layer avec un Controller Layer en backend de l'API REST
- Configurer des requêtes préparées dans le backend de l'API avec des paramètres dont le type et le format sont validés dans le Controller Layer pour éliminer les risques d'injection
- Implémenter un principe de moindre privilège avec une séparation des rôles au niveau du serveur qui expose l'API défini comme suit :
 - Rôle lecteur seul : les routes de l'API exposées au client permettent de requêter la base de données avec des requêtes préparées. Sur ces routes il y

a besoin d'un bearer token à validité limitée (oauth2). Le token doit être envoyé avec chaque requête GET. Le token est généré au niveau de notre serveur API.

- Rôle administrateur (incluant les accès lecteur): certaines routes par exemple celle qui permet d'exécuter le pipeline ETL avec l'API (1 seule exécution non orchestrée pour ajouter des données ciblées) a besoin d'une autorisation ADMIN. Avec ce rôle, l'utilisateur peut exécuter l'ETL avec un nombre limité de lignes à obtenir en output (à cause du timeout, de la consommation des ressources de calcul et de la facturation). A ce niveau c'est le package **dpe_enedis_ademe_etl_engine** qui se connecte et écrit dans la base de données les nouvelles lignes. Le client de l'API (même ADMIN) n'a pas d'accès d'écriture sur la bdd directement. En revanche, l'ADMIN a accès à l'interface de monitoring de l'ETL.
- Implémenter des rate limiteurs sur les routes qui permettent d'obtenir la donnée (1 requête/seconde par adresse IP).

Les données exposées par les routes de l'API dans notre SI ne contiendront à aucun moment les informations complètes. Les données exposées sont celles à la fin du traitement (données gold) dont les identifiants auront été retirés. La table des adresses ne sera jamais exposée au client. Au lieu de cela, les adresses seront chiffrées et il y a un système de recherche par similarité pour préserver l'anonymat complet des données.

Enfin, pour la gestion des incidents de sécurité et la cohérence des données exposées, il faudra lever des alertes avec des issues dans le repos github de l'API (dans la partie github project) d'une part et d'autre part s'assurer en continu que les sources de données utilisées respectent les droits des personnes concernées en termes de données personnelles (veille technologique sur les textes de loi du RGPD).

3- Schéma d'architecture de sécurité

- *Présentation de l'architecture mise en place pour la sécurité des données*

Comme présenté dans la partie précédente, pour la persistance des données, les résultats de l'ETL sont stockés dans la base de données et nous utilisons une API REST pour exposer cette base de données au client avec un système d'authentification à chaque requête sur les routes pour lire des données (voir Figure 10). Ce choix de conception permet de mettre plusieurs couches de sécurité autour de la source de données et d'exposer une interface de requête unique à l'utilisateur avec des niveaux d'accès. Les rôles ont été présentés dans la partie précédente.

L'API est conçue de sorte à servir de data access layer. En effet, les utilisateurs authentifiés (administrateur ou lecteur) peuvent requêter des données dans le serveur de la base de données, peuvent être également redirigés vers les autres micro-services qui composent le serveur à savoir : le filestorage S3 et l'interface de monitoring UI du serveur prefect. Toutes les routes qui permettent de requêter la base de données ont des rate-limiteur (Figure 20 : Swagger UI du serveur API). Le schéma ci-dessous montre l'interaction entre les différents services et les flux d'échange des données.

Le serveur de l'application dont ce projet fait l'objet est composé de 5 services qui échangent des données. Il s'agit du module API SERVER FASTAPI qui encapsule l'application ETL, du module SERVEUR POSTGRES de base de données (pour la persistance), du module de CACHE REDIS, du module de FILESTORAGE S3 et du module de MONITORING via le serveur prefect. Le module API SERVER sert de point d'entrée et de data access layer pour le serveur. Il gère l'authentification, la validation des données et expose des routes pour soit récupérer la donnée, uploader des fichiers ou rediriger vers d'autres modules (module de monitoring par exemple). Il encapsule l'ETL et permet de déclencher (avec planification ou non) l'exécution de l'ETL. L'ETL en est un sous module (à installer via le gestionnaire de packages ou à importer la base de code).

A la fin du pipeline de transformation dans l'ETL, la donnée « gold » doit être chargée sur un SERVEUR POSTGRES pour la persistance et éviter de réexécuter la charge de travail à nouveau. Pour permettre d'implémenter un certain nombre de pratiques en lien avec la sécurité et l'utilisation inefficace de puissance de calcul à chaque nouvel utilisateur, on utilise un système de CACHING dans l'outil de dataviz. Un service tiers de CACHING est également utilisé pour gérer la connexion via des One Time Passwords (OTP) et fixer un rate-limiteur sur les routes de l'API pour récupérer de la donnée afin de se prémunir des attaques DDOS. Le filestorage représente le module S3 qui stocke les fichiers temporaires et les zones (bronze, silver, gold) de l'ETL. Le monitoring du pipeline, les logs applicatifs et le profiling des fonctions de l'ETL se fait grâce au serveur prefect.

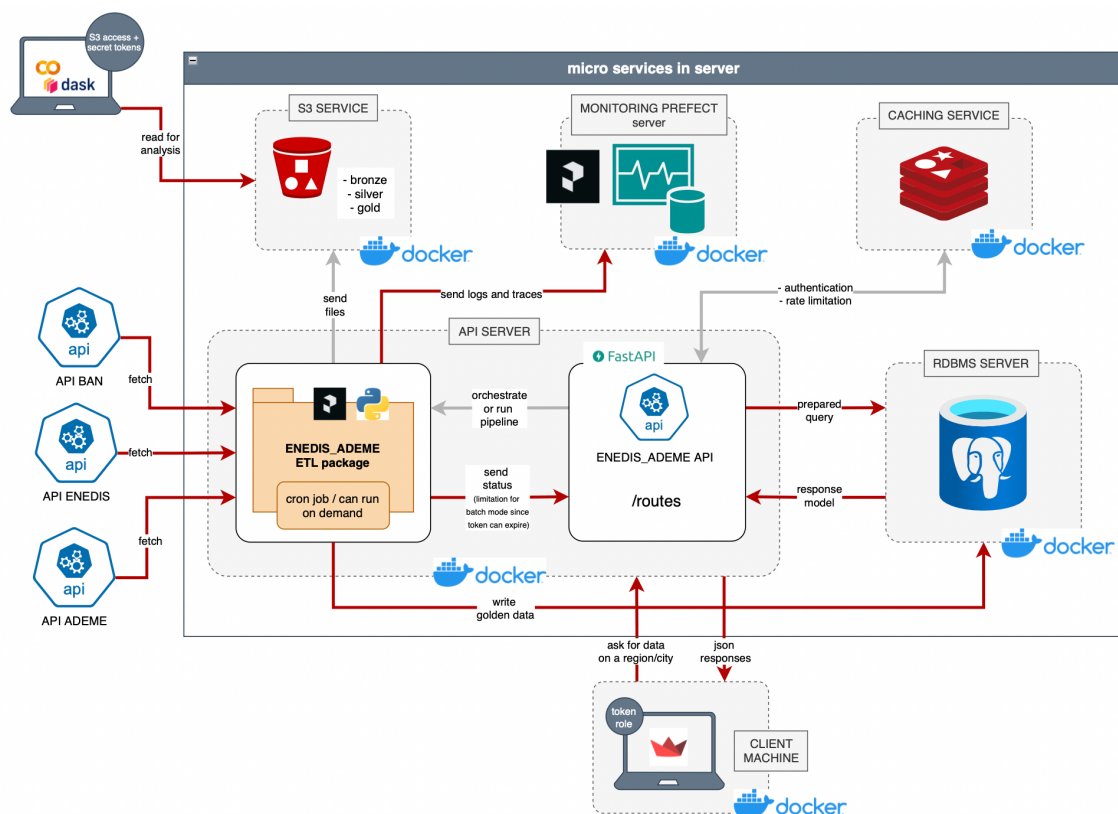


Figure 9 : Flux d'échange de données entre les services

- Identification des zones d'attention

Les points d'attention au regard de cette architecture sont les suivants :

- Communication client-serveur
 - Risques de DDOS : requêtes multiples adressées à une même route pour surcharger le serveur
 - Temps de réponse : time-out de l'API, une charge de travail trop importante alourdirait la facture
 - Injections SQL dans les routes de l'API
 - Limitations pour les extractions importantes (batch utiliser l'orchestration)
 - Les sources de données : vérifier que les APIs externes renvoient toujours le même schéma de réponse
 - L'accès aux données silver depuis le S3 par les Data scientist pour l'analyse : requiert un secret key et access key.
- *Mise en œuvre des solutions de protection des données*

Pour l'authentification, la communication entre l'application cliente de visualisation et le serveur est faite au moyen du protocole http. A chaque nouvelle connexion sur l'outil de dataviz, l'utilisateur devra s'authentifier avec un OTP que nous lui envoyons par email. Ensuite on lui envoie un bearer token à durée limitée à utiliser dans les headers de chaque requête http. Ceci permet de gérer les autorisations. Enfin on a un système de gestion de rôle avec des niveaux accès différents pour chaque route (certaines routes sont accessibles par les utilisateurs authentifiés en tant qu'admin et d'autres sont justes accessibles aux lecteurs).

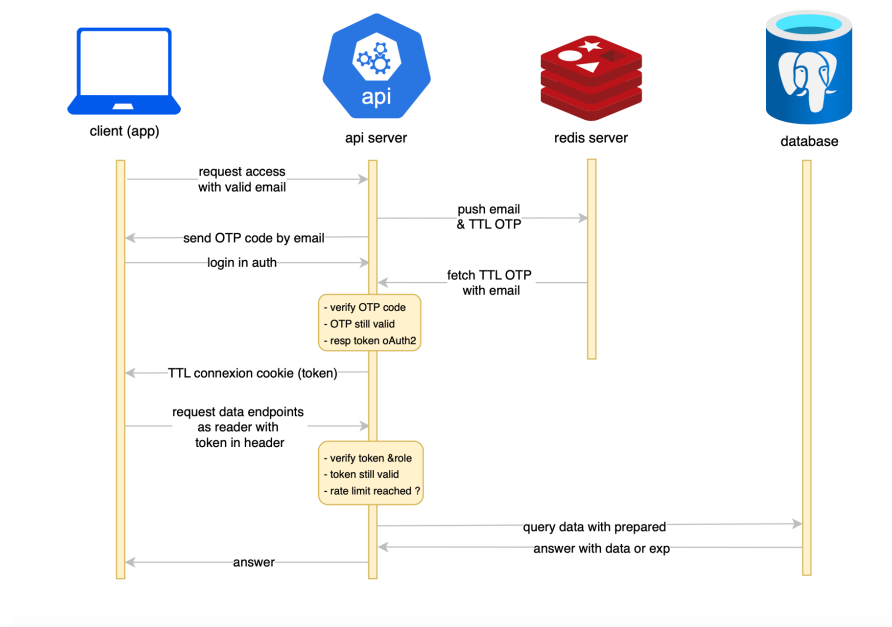


Figure 10 : Sécurité de la base de données – authentification par token

Les autres solutions concernent le backend de l'API et ont été implémentées dans le code. Le résultat peut être vérifié sur les images du swagger UI (Figure 20 et 21) de l'API et sur le dépôt github du projet.

Sources

- Origine du projet : <https://defis.data.gouv.fr/defis/diagnostics-de-performance-energetique>
- Open data : [licence ouverte Etalab](#)
- <https://energy-manager.readthedocs.io/en/latest/>, projet traitant du même sujet identifié sur pypi et github
- SQL MAP – outil de test de vulnérabilité contre les injections SQL : <https://pypi.org/project/sqlmap-websocket-proxy/>

Annexes

- *Documentation - Dépôts GitHub du projet et git modules*
 - o [DPE-Energy-Performance-Analysis](#)
 - o [Submodule DPE-Energy-Performance-Analysis-API](#)
 - o [Submodule DPE-Energy-Performance-Analysis-ETL](#)
 - o [Submodule DPE-Energy-Performance-Analysis-ClientApp](#)
- *Calculs estimations volume des données*

Dans les lignes qui suivent, quelques estimations sur le volume de données à traiter sont réalisées.

- Pour les DPE ADEME

Sachant qu'il y a 6 millions de DPE et qu'un DPE compte environs 200 variables réparties comme suit : environs 100 variables de type chaîne de caractères, environs 100 de type int64 ou float64. L'espace occupé par 1 seul DPE peut être estimé. Puisque 1 caractère dans une chaîne occupe environs 1 octet, que 64 bits font 8 octets et que 1 024 octets font 1 Ko :

$$(20 \text{ octets en moyenne/chaîne} * 100 \text{ variables}) + (8 \text{ octets} * 100 \text{ variables}) = 2\,800 \text{ octets}$$

Donc 1 seul DPE occuperait environs en moyenne 3 Ko. La totalité de la base DPE occuperait alors environs 18 Go. Autre approche : en re-typant toute l'info en chaîne de caractère dès le début, on obtient environs 14 Go si on suppose que les nombres font 5 caractères en moyenne.

- Pour les consommations Enedis

Chaque année, il y a :

$$400\text{milles adresses} \times \text{environs } 10\text{octets/variable} \times 20\text{variables} = \text{environs } 0.08 \text{ Giga octets/an}$$

- Pour les identifiants de la BAN

Il s'agit juste d'un code alpha numérique d'une dizaine de caractères à ajouter aux deux sources de données pour établir la connexion. Donc les résultats ne sont pas significativement impactés.

- Figures et captures d'écrans

```

tool = DataEnedisAdeme('../ressources/data/enedis_2018.csv')
tool.extract()

# avec un sample de 1005 lignes
# WARNING:root:Loading from :
# ../ressources/data/enedis_2018.csv
# WARNING:root:Valid data BAN : 1004/1005
# WARNING:root:Extraction results : 7574 rows, 280 columns. => 4min

# -----
# scope de cette extraction : PARIS 2018 code département = 75 et année = 2018
# 50k lignes de enedis => 50k adresses BAN mappées
# en croisant avec ademe : on a 465_961 lignes asachant q'une adresse enedis regroupe plusieurs logements ademe
# --> extract sauvegardé

[30] ✓ 249m 53.8s Python

```

WARNING:root:Loading from :
 ../ressources/data/enedis_2018.csv
 WARNING:root:Valid data BAN : 50026/50061
 WARNING:root:Extraction results : 465961 rows, 280 columns.

	Conso_chauffage_dépensier_é_finale_ademe	Volume_stockage_générateur_ECS_n°1_ademe	Conso_é_finale_installation_ECS_ademe	Nom_commune_(BAN)_ademe	Emis
0	16170.8	0.0	1548.3	Paris	
1	20081.5	0.0	5617.0	Paris	
2	38501.7	0.0	5812.1	Paris	

3 rows x 280 columns

Figure 11: Aperçu résultat de la collecte en mode batch (+ de 10_000 PDL Enedis)

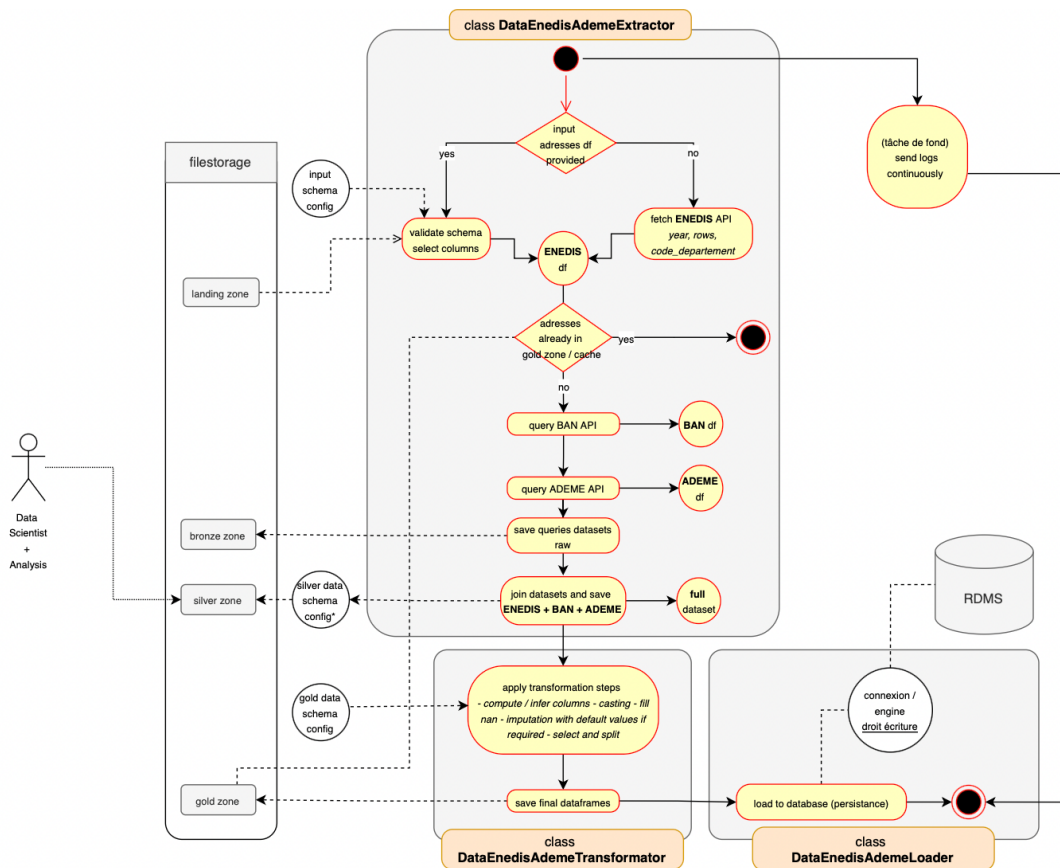


Figure 12: Schéma UML du fonctionnement de l'ETL

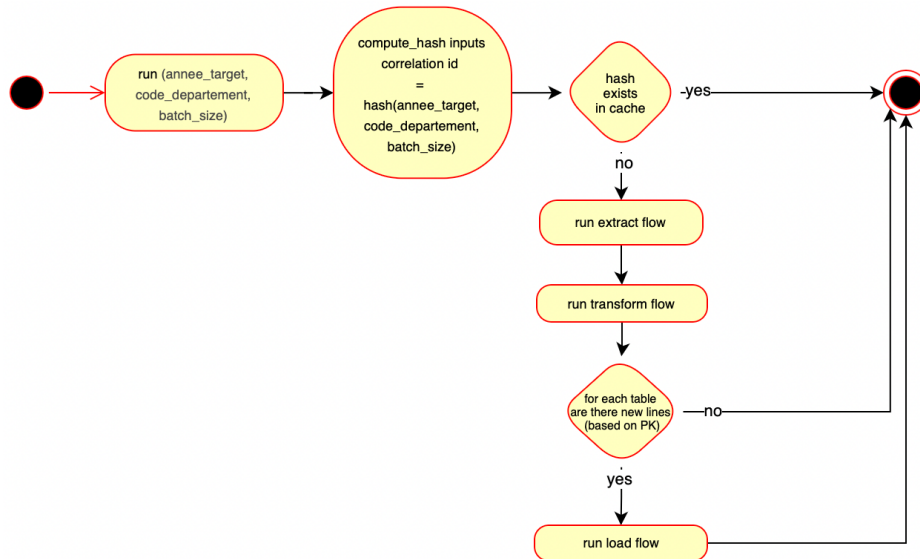


Figure 13 : Stratégie de mise à jour, reprise sur erreur et idempotence

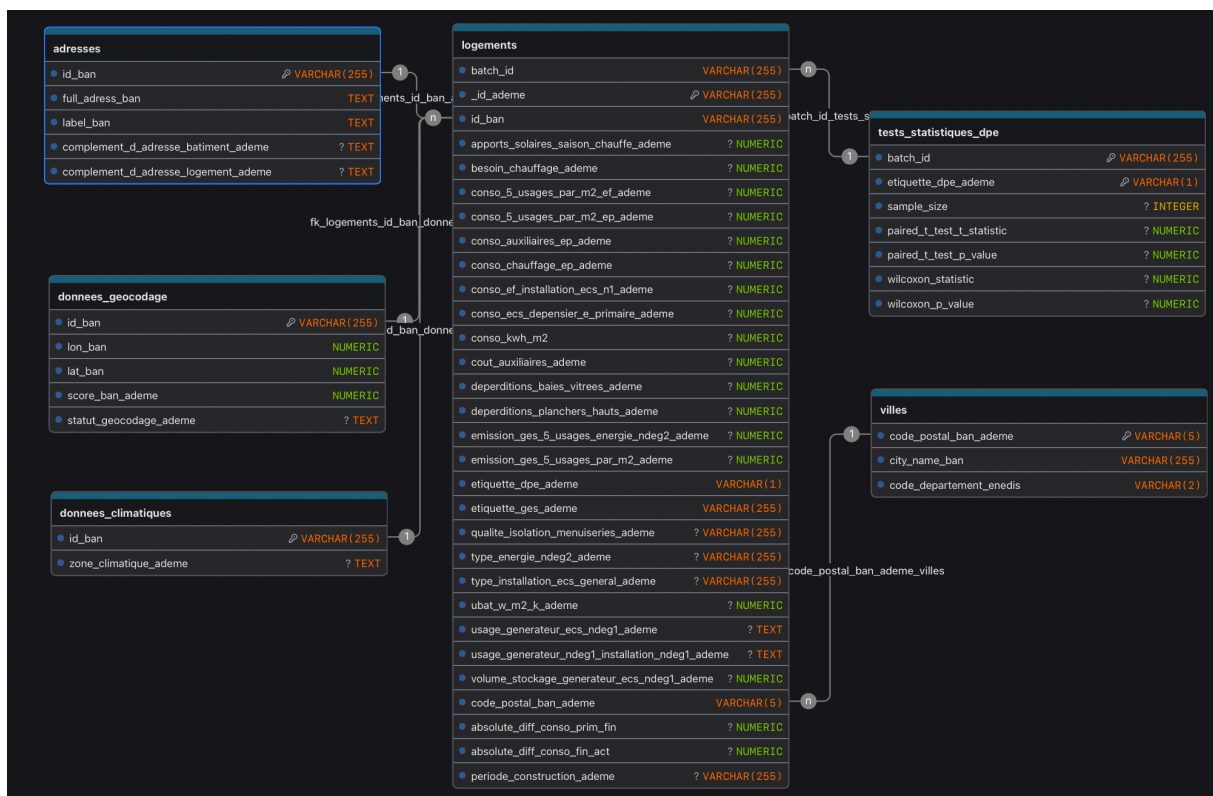


Figure 14 : Schéma de la base de données - zone gold

```

1  TACHE EXTRACTION 1
2  @decorator_logger
3  def get_enedis_data(
4      self,
5      from_input:bool=False,
6      code_departement:int=75,
7      annee:int=2022,
8      rows:int=10
9  ):
10     """
11     Extraire le dataframe enedis soit à partir d'un fichier csv
12     soit à partir d'une url.
13     :param from_input: If True, use the input CSV file. If False, use the Enedis API.
14     :param code_departement: Code of the department to filter the data.
15     :param annee: Year to filter the data.
16     :param rows: Number of rows to extract from the Enedis API.
17     :return: self, with self.input containing the Enedis data.
18     """
19     def add_full_adress_col(df):
20         df['full_adress'] = df['adresse'] + ' ' + df['code_commune'] + ' ' + df['nom_commune']
21         return df
22
23     if from_input:
24         self.load_batch_input()
25         if self.debug: self.debugger.update({'source_enedis': "input csv"})
26         if code_departement: # filter sur le code département dans le df input
27             logger.info(f"Filtering input data on code département : {code_departement}")
28             self.input = self.input[self.input['Code Département']==code_departement]
29     else:
30         requete_url_enedis = self.get_url_enedis_year_rows(annee, rows)
31         if code_departement: # filter sur le code département dans l'url
32             requete_url_enedis += f"&where=code.departement = {code_departement}"
33         self.input = self.get_dataframe_from_url(requete_url_enedis)
34         logger.info(f"Extract input from url enedis : \n {requete_url_enedis}")
35         if self.debug: self.debugger.update({'source_enedis': requete_url_enedis})
36
37     # enfin, reconstituer les adresses complètes
38     self.input = add_full_adress_col(self.input)
39     return self

```

Figure 15: Code snippet - Tâche collecte de données Enedis

```

1  # TACHE EXTRACTION 2
2  @decorator_logger
3  @task(name="extract-data-from-ban-api", retries=3, retry_delay_seconds=10, cache_policy=NO_CACHE)
4  def get_ban_data(self, n_threads):
5      """
6      Extraire le dataframe de la BAN à partir d'une liste d'adresses.
7      :param n_threads: Number of threads to use for querying the BAN API.
8      :return: self, with self.ban_data containing the BAN data.
9      :raises ValueError: If the input dataframe is empty.
10     """
11     logger = get_run_logger()
12     # tache 1 - prendre input enedis
13     if self.input.empty:
14         logger.critical("Erreur dans le chargement du fichier CSV input : pas de données")
15         raise ValueError("Pas de données dans le dataframe")
16
17     # tache 2 - constituer les adresses enedis
18     enedis_adresses_list = list(set(self.input.full_adress.values.tolist()))
19     if self.debug: print(f"> get_ban_data : {len(enedis_adresses_list)}")
20
21     self.ban_data = self.multithreaded_api_request(
22         num_threads=n_threads,
23         api_call_func=self.call_ban_api_individually,
24         obj_list=enedis_adresses_list,
25         rate_limit=30 # 50 en vrai d'après la doc
26     )
27
28     # tache 4 - filtrer les adresses valides
29     self.ban_data = list(filter(lambda x: x is not None, self.ban_data))
30     if not self.ban_data:
31         logger.critical("Erreur dans le chargement des données BAN : pas de données")
32         raise ValueError("Pas de données dans le dataframe BAN")
33
34     # tache 5 - convertir en dataframe pandas
35     self.ban_data = pd.DataFrame(self.ban_data)
36
37     vectorized_upper = np.vectorize(str.upper, cache=True) # est une optimisation
38     self.ban_data['label'] = vectorized_upper(self.ban_data['label'].values)
39     # on remet en upper car on a besoin pour le merge avec enedis
40     if self.debug: self.debugger.update({'sample_ban_data': self.ban_data.tail(5)})
41     logger.info(f"Valid data BAN : {len(self.ban_data)} addresses founded over {len(enedis_adresses_list)} requested.")
42     return self

```

Figure 16: Code snippet- Tâche requête de géocoding BAN

```

1 # TACHE EXTRACTION 3
2 @decorator_logger
3 def get_ademe_data(self, n_threads):
4     """
5     Extraire la data de l'ademe au complet en utilisant
6     la liste des id_ban.
7
8     - recup le df ademe data sur la base des id_ban
9     - sur la base des Identifiants BAN de enedis, aller chercher les logements mappés sur ces codes BAN
10    - 1 id_ban = * adresses (entre 10 et 1_000) - en effet, les données enedis sont agrégées
11    """
12    ademe_data = []
13    # ? multithreading -> limite les requetes en parallele - renvoie 0 resultats si trop de requetes en parallele
14    ademe_data_res = [requests.get(self.get_url_ademe_filter_on_ban(id)).json().get('results')\
15                      for id in self.id_BAN_list]
16
17    # suppr les None
18    ademe_data_res = list(filter(lambda x: x is not None, ademe_data_res))
19    if not ademe_data_res:
20        logger.critical("Erreur dans le chargement des données Ademe : pas de données")
21        raise ValueError("Pas de données dans le dataframe Ademe")
22    # on a une liste de listes, chaque liste correspond à un id_ban
23    # on obtient une liste à 2 niveaux pour chaque Id_BAN
24    for _ in ademe_data_res:
25        ademe_data.extend(_)
26    del ademe_data_res
27    ademe_data = pd.DataFrame(ademe_data)
28    ademe_data = ademe_data.add_suffix('_ademe')
29
30    self.save_parquet_file(
31        df=ademe_data,
32        dir=self.PATH_DATA_BRONZE,
33        fname="ademe_data_tmp.parquet"
34    )
35    self.ademe_data = ademe_data.copy()
36    del ademe_data
37    return self

```

Figure 17: Code snippet - Tâche requête API ADEME

```

1 # TACHE MERGE 1
2 @decorator_logger
3 def merge_and_save_enedis_with_ban_as_output(self):
4     """obtenir le df pandas des données enedis (requete) + les données de la BAN pour les adresses trouvées, ""
5     # note : si une adresse est pas trouvée on te j la data enedis (cf. inner join)
6     # merge enedis avec ban
7     # ? - free memory
8     self.input = self.input.add_suffix('_enedis')
9     self.ban_data = self.ban_data.add_suffix('_ban')
10    self.output = pd.merge(
11        self.input,
12        self.ban_data,
13        how='inner',
14        left_on='full_address_enedis',
15        right_on='full_address_ban'\
16        ,rename(columns={'id_ban': 'id_BAN'})
17    )
18    self.id_BAN_list = self.output.id_BAN.values.tolist()
19    if self.debug: self.debugger.update({'sample_output_enedis_with_ban_tmp': self.output.tail(5)})
20    if self.debug: self.debugger.update({'id_BAN_list': self.id_BAN_list})
21    self.save_parquet_file(
22        df=self.output,
23        dir=self.PATH_DATA_BRONZE,
24        fname=f"enedis_with_ban_data_tmp_{get_today_date()}.parquet"
25    )
26    self.output = pd.DataFrame() # free memory
27    self.ban_data = pd.DataFrame() # free memory
28    return self
29
30 # TACHE MERGE 2 (final)
31 @decorator_logger
32 def merge_all_as_output(self):
33     """
34     Merge all dataframes to get the final dataframe.
35     """
36     # reconstituer le dataframe complet
37     enedis_with_ban_data = self.load_parquet_file(
38         dir=self.PATH_DATA_BRONZE,
39         fname=f"enedis_with_ban_data_tmp_{get_today_date()}.parquet"
40     )
41     # enedis_with_ban_data = enedis_with_ban_data.add_suffix('_enedis_with_ban')
42     logger.info(f"Enedis with BAN data loaded : {enedis_with_ban_data.shape[0]} rows, {enedis_with_ban_data.shape[1]} columns.")
43     logger.info(f"Ademe data loaded : {self.ademe_data.shape[0]} rows, {self.ademe_data.shape[1]} columns.")
44     assert 'Identifiant_BAN_ademe' in self.ademe_data.columns, \
45         "Identifiant_BAN_ademe column not found in Ademe data. Check the schema or the data extraction process."
46     assert 'id_BAN' in enedis_with_ban_data.columns, \
47         "id_BAN column not found in Enedis with BAN data. Check the schema or the data extraction process."
48     # merge enedis with ban data and ademe data
49     self.ademe_data['Identifiant_BAN_ademe'] = self.ademe_data['Identifiant_BAN_ademe'].astype('string')
50     enedis_with_ban_data['id_BAN'] = enedis_with_ban_data['id_BAN'].astype('string')
51     self.output = pd.merge(self.ademe_data,
52                            enedis_with_ban_data,
53                            how='left',
54                            left_on='Identifiant_BAN_ademe',
55                            right_on='id_BAN').drop_duplicates().reset_index(drop=True)
56     # normaliser les noms de colonnes et trier les colonnes
57     self.output = normalize_df_colnames(self.output)
58     self.save_parquet_file(
59         df=self.output,
60         dir=self.PATH_DATA_SILVER,
61         fname=f"extract_output_data_{get_today_date()}.parquet"
62     )
63     if self.debug: self.debugger.update({'sample_output': self.output.tail(5)})
64
65 @decorator_logger
66 def extract(self,
67             from_input_tool=False,
68             input_csv_path:str=None,
69             code_departement:int=75,
70             annee:int=2022,
71             rows:int=10,

```

Figure 18: Code snippet - Tâches de jointures, extraction

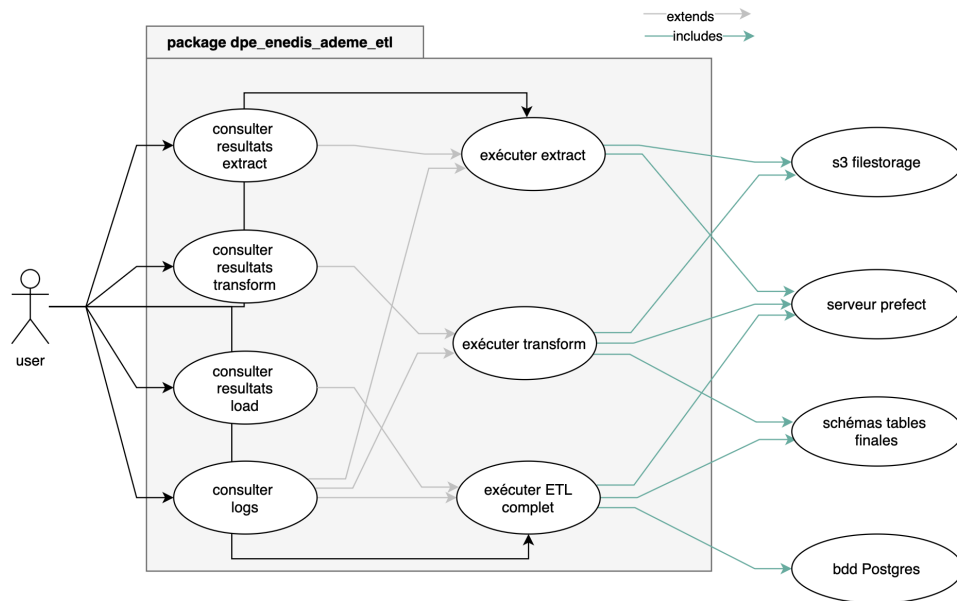


Figure 19 : Diagramme de cas d'utilisation package ETL

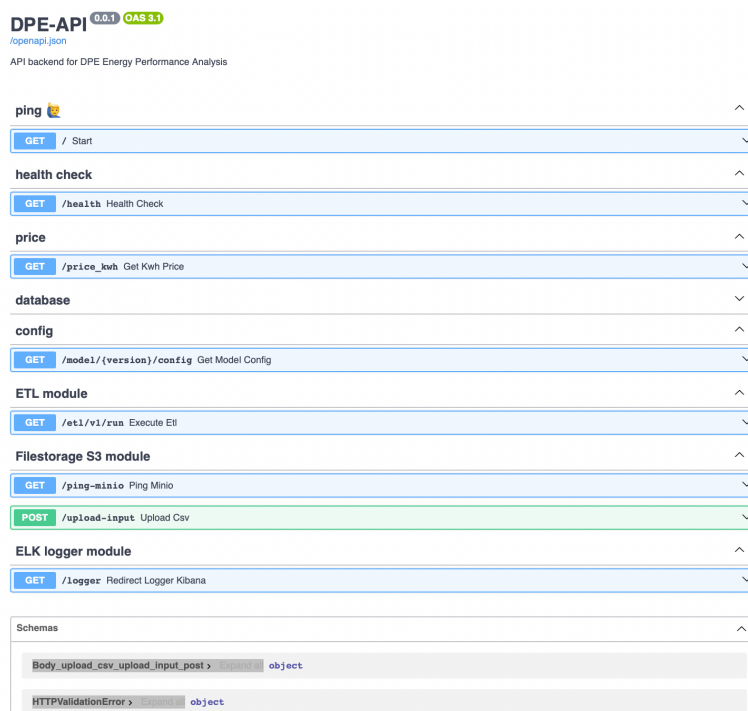
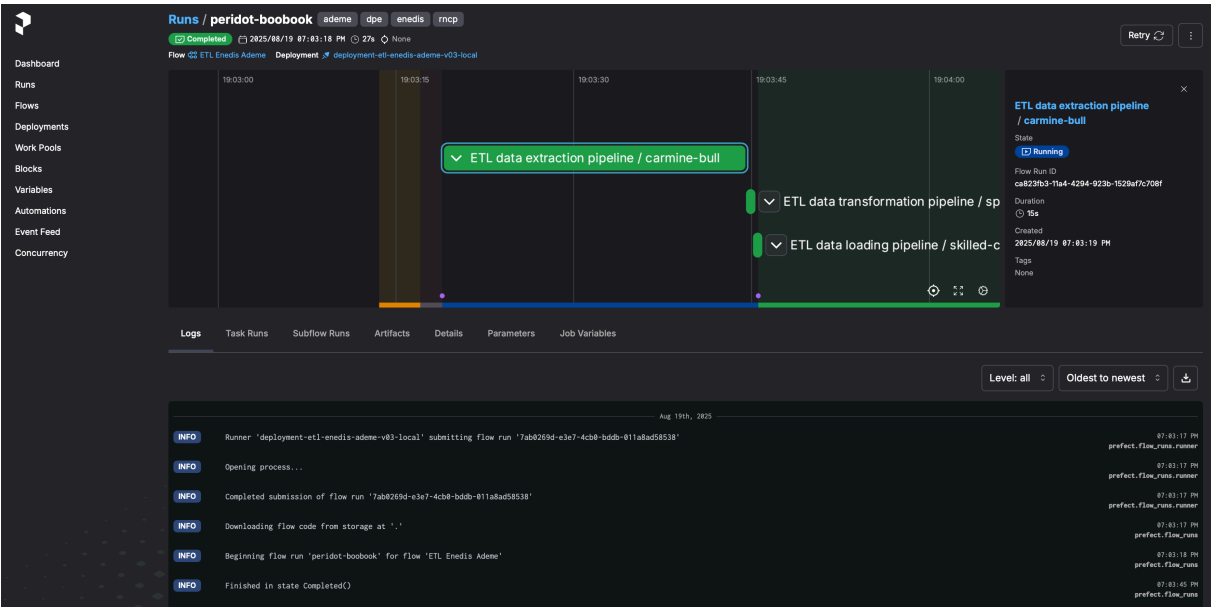
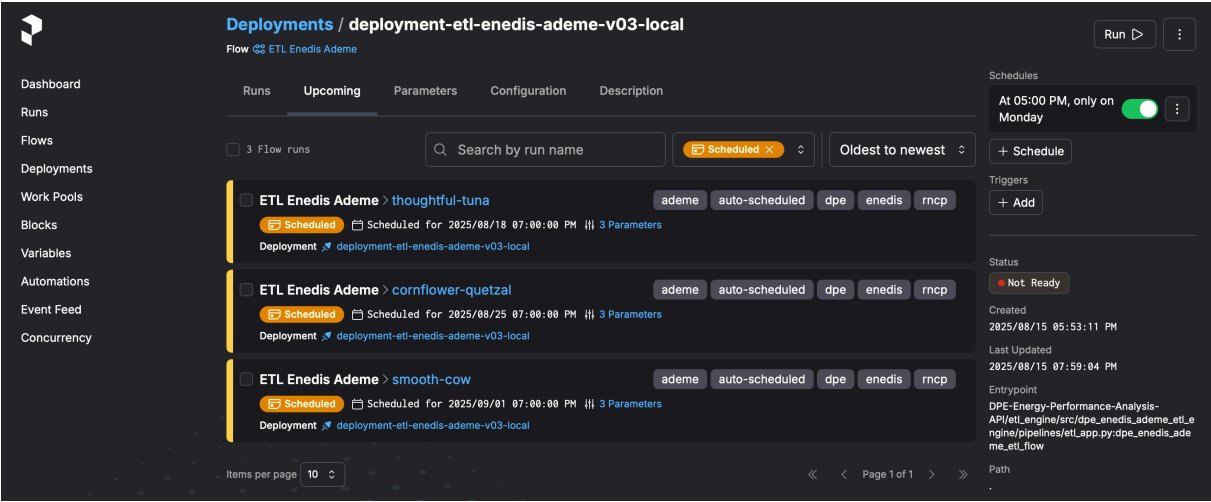


Figure 20 : Swagger UI du serveur API

GET	/price_kwh	Get Kwh Price	
auth			
POST	/send-otp	Send Otp	
POST	/login-with-otp	Login With Otp	
GET	/user-status	Get User	
database			
GET	/db	Init Db Require Admin Level Access	
GET	/db/reader/addresses/getall	Get All Addresses Require Admin Level Access	
GET	/db/reader/addresses/searchaddress/{searched}	Search Similar Addresses Require Reader Level Access	
GET	/db/reader/addresses/getone/{searched_address}	Get One Addresses Require Reader Level Access	
GET	/db/reader/addresses/cities	Get Cities Names And Codes Reader Level	
GET	/db/reader/addresses/{city_name}/allcoords	Get Coord By City Name Reader Level	
GET	/db/reader/logements/getall	Get All Logements Reader Level	
GET	/db/reader/logements/getbyadress/{searched}	Get Logements By Adress Reader Level	
GET	/db/reader/logements/getonebycity/{city_name}	Get One Logements By City Name	
GET	/db/reader/logements/getallbycity/{city_name}	Get All Logements By City Name	
GET	/model/{version}/config	Get Model Config	



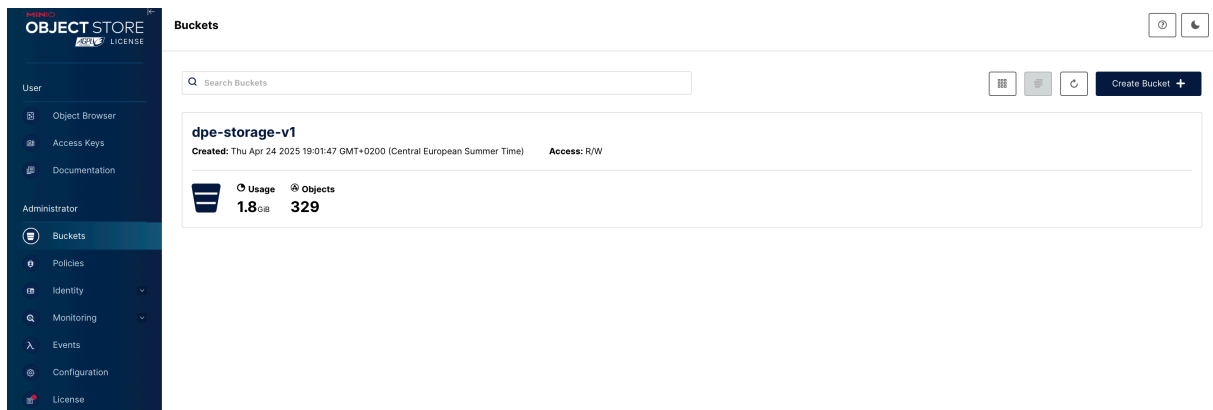


Figure 24 : MinIO UI - 1

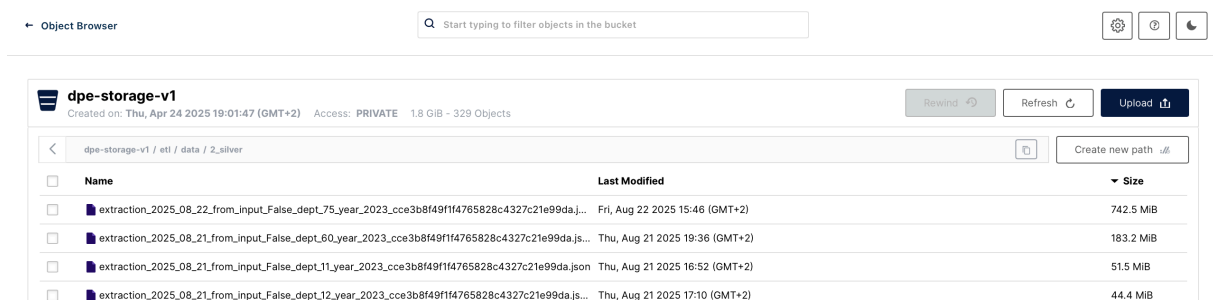


Figure 25 : MinIO UI - 2